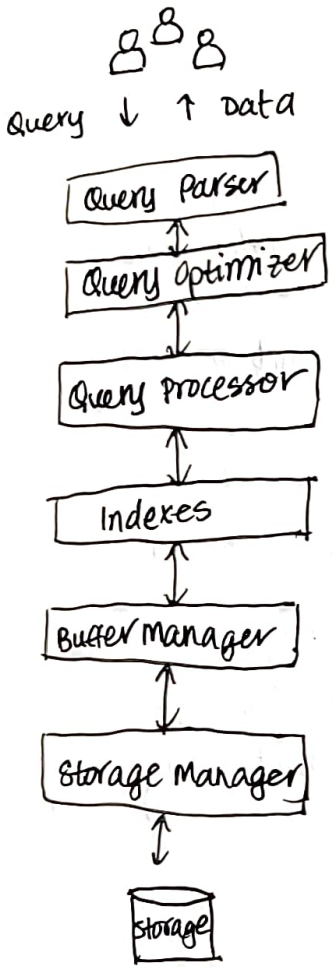
8.1 Accessing and Indexing Data

Querying a Database: The "Query-To-Data" Stack

- Query Parsing:
  - Query parser parses the query
- Query Optimization:
  - Query optimizer generates a "query plan"
- Query Processing:
  - Query processor executes the plan
- Data Indexing:
  - Indexes provide "maps" to where data is in storage.
- Data Accessing:
  - Buffer and storage manager manage I/O between storage and memory.



Accessing Data: Getting to our Data!

- Data is large
  - Main memory is limited
- Data must be persistent
  - Main memory is volatile
- So, "big data" is stored in external storage.
- To answer a query, we must bring data to memory.
- Buffer Manager:
  - Deals with "buffering" data in memory for processing.
- Storage Manager:
  - Deals with reading and writing data to storage.

Disk to main memory

Indexing Data: Mapping our Data!

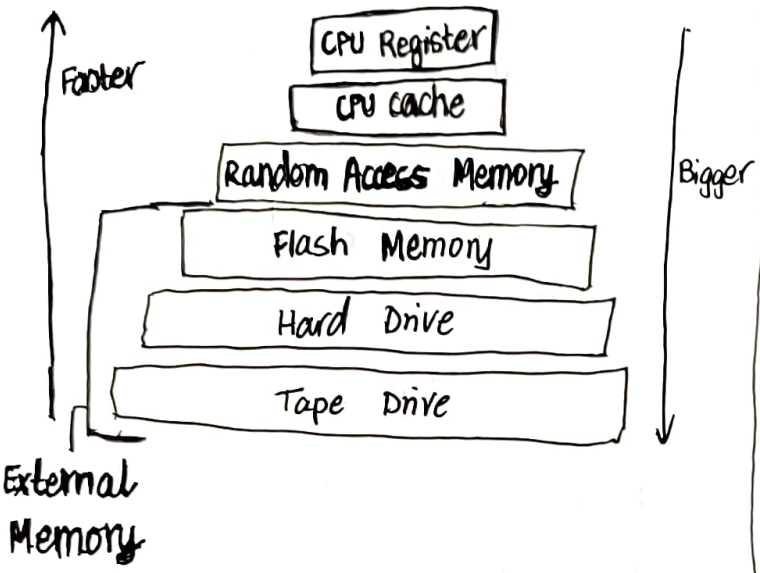
- Data stored in storage is large
- Data we need for querying is often a small fraction.
- How can we find necessary data quickly?
- Query processor needs "maps" to locate data.
  - ... without having to scan the entire database.
- An index provides such a "map" for locating data.
  - By values, such as grade 80 or course "CS 411".

8.2 Accessing Data

Where is Data? No Longer all in Memory

- Data cannot fit in main memory. It is big!
  - No matter how main memory grows, data grows too.
- Must utilize memory hierarchy – with external memories.

Memory Hierarchy



Storing Data in External Storage

- Storage = Array of Blocks
- Unit of storage allocation/access: Block
  - Name not quite standardized – "page", "cluster" too.
  - Each block has a fixed size depending on the file system/OS.
  - Usually 1K, 4K, 8K bytes.
- Table is stored on a set of blocks
  - Contiguous or not.

Storage: The evolution of Hard Disk Drive

1956	2017
3.75 Mb	1 TB
68 ft^3 / 1.9 m^3	fits in computer
600 millisecond access time	8.5 ms
$9,200 per Mb	~ $100
	14.11 oz

How is data stored in blocks?

Row or columnar storage.

ex. Oracle DB

Accessing Data on Disks: Where does the time go?

- Seek time: Time for disk head to move to the track of the block.
  current: ~ 10ms
- Rotational Latency: Time for first sector of the block to rotate under the head.
  current: ~ 5ms
- Transfer Time: Time to read or write data to the sectors.
  current: ~ 200 MB/s or 0.02 ms for 4KB

Disk with hard surface hence Hard Drive

A few sectors = a block.

1) Disk head moves to track (seek time)
2) Disk rotates until target sector is under disk head. (rotational latency)
3) Read/write all the sectors/blocks. (Transfer time)

time ∝ data size, no. sectors/blocks.

Accessing Data on Disk: There are 2 modes

- Random Access
  - Access data randomly - at arbitrary positions.
  - Get(address) → Block
  - Cost = Seek Time + Rotational Latency + Transfer Time
  - For 4KB block: 10+5+0.02 ~15ms
- Sequential Access
  - Access data sequentially - in a predetermined ordered sequence.
  - GetNext() → Block
  - Cost = Transfer Time
  - For 4KB block: 0.02ms

8.3 Indexing Data

Why Indexing? Where is Data?

- DBMS should answer queries quickly -- many challenges.
- First of all, where is data? Each query may focus on some part of the database, and we need to locate it.

Regardless of Data Models or Query languages: Need to find Data to Answer Queries -- by Values.

- A query specifies data of interest by values in the "WHERE" conditions (or the like)
- Find records (tuples, documents or nodes) that match certain values, or search keys.

How Do We Find Target Data? (54)

- Q1: Find students who scored 80 in CS411.

0	Bugs Bunny	CS411	90	Blocks (4)
	Donald Duck	BIO300	92	
1	Donald Duck	CS423	93	
	Donald Duck	CS411	95	
2	Bugs Bunny	CS423	80	
	Mickey Mouse	CS423	70	
3	Peter Pan	CS411	94	
	Charlie Brown	ECON101	50	

Method 1: Table Scan

- Suppose table is stored in an unordered set of blocks.
  - called a heap file.
- Q1: Find students who scored 80 in CS411.
- Go to where file starts.
- Read next block. Process it.
  - Keep the tuples whose grade = 80 and course = "CS411".
- Until the end of file.
- Cost = O(B), for a file of B blocks.

Method 2: Binary search on sorted Files

- Suppose table is sorted on disk by the target attribute (grade).
  - called a sorted file.
- Q1: Find students who scored 80 in CS411.
- Perform binary search to find target values.
  - Locate grade = 80.
- Check the remaining conditions.
  - check course = "CS411"
- Cost = $O(\log_2(B))$

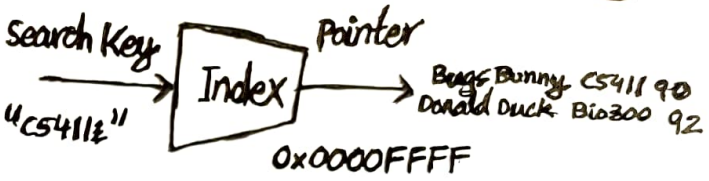
Can we do better?

If the table occupies B blocks:

- Table Scan:
  - Access B blocks.
- Binary search on sorted Files:
  - We can only sort the table in one order.
    - If by grade then not by course.
  - Even when sorted, binary search is still expensive.
    - $\log_2(B)$ blocks.

Indexing: What is an Index?

- Indexing is the action of building an index.
- Index: A data structure that provides a mapping from a search key value to the locations, or pointers, of data that matches the key.
- search key = any subset of the fields of a relation.
  - search key is not the same as key (minimal set of fields uniquely identifying a record)
- It speeds up query processing.
  - so we can quickly locate where data is.
  - So we do not need to scan through all records in the database.



Indexes in a Database

- DBMS auto manages or lets users manage indexes.
- DBMS uses indexes to speed up query processing.

What are the desired properties?

- Fast, of course.
- Scalable to larger data or more users.

8.4 ISAM Index for static Data

Index sequential Access Method (ISAM)

Index sorted File of Blocks

~~How do we~~

- We want to create ISAM index for the Enrolls table on the grade attribute.
- How do we create "pointers" to search key values (grades) in the sorted file?

To Index: We ask- What pointers shall we store?

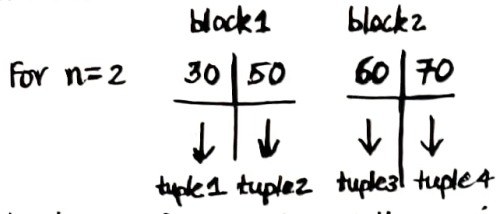
- An index is to provide "directions" -i.e. pointers to data.
  - An index entry is much like a street sign.
- Thus, our objective is to store a collection of pointers.
  - Then we use keys to label these pointers to follow.
- Let's start with - having a pointer to every key value.
  - This is called a dense index.
  - other choices: to every block...

How Do We store these pointers? How to label them?

Our "signs": Index Blocks

Case 1: - for Pointer to a specific key

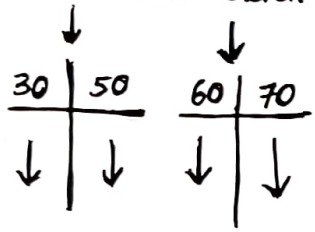
- Just label the pointer with the key value: i.e. (K, P)
- store n pointers and their n keys in a disk block.



- To find tuple of Key K_i, follow pointer P_i.
- Typical in a tree index (e.g. ISAM, B+ tree) as leaf node structure.
- How large is n? As many as a block can fit.

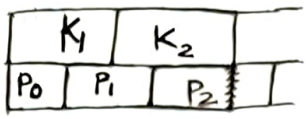
Create one Level of Index Blocks

- This is useful, but not very useful (yet)
- can we create one more level, to index these index nodes?



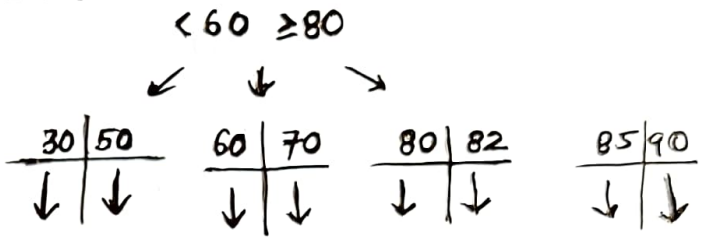
How do we store and label the pointers? case 2 - for pointer to a set of keys

- Each pointer leads to a set of keys
- We will label each pointer by a range $[K_i, K_{i+1})$.
- An index block contains n keys and n+1 pointers.
- To find tuples of key in $[K_i, K_{i+1})$, follow pointer P_i.
- Typical in a tree index (e.g. ISAM, B+ tree) as non-leaf node structure.
- The number of pointers is the fanout $F = n+1$.



Add Next Level of Index Blocks

- Add index blocks to point to the last level.



Add one More Level and Done

ISAM - Indexed Sequential Access Method

- Add one level of index blocks to point to data blocks (leaf nodes).
- Add one level of index blocks to point to the previous level (non-leaf nodes).

.....

- Until there is only one index block (root)

ISAM: Lookup

- Lookup(key) → pointer to tuple
  - Start at the root
  - Follow pointers by the ranges they represent.
  - Until reach a leaf node
  - Check if key is there.
- Cost: Number of pointer hops to reach a leaf
  - Each pointer hop is a random access to fetch a disk block.
  - Number of pointer hops = height of tree h.
  - For indexing N tuples, with fanout F, $N \leq F^h$, or $h = \lceil \log_F(N) \rceil$.
- E.g., N = 1000000, F = 341: cost = $h = \lceil \log_{341}(1000000) \rceil = 3$

ISAM: How to insert/delete new data?

- Insert into vacancy or overflow blocks.
- Delete just removes keys

When data is dynamic you get an uneven ISAM index.
Disk utilization can be bad because some nodes can become empty whilst others become very full

~~Could~~ can you use ISAM for unsorted data?